

DevOps Utilities API – Complete Code Explanation

12 May 2026 01:13

DevOps Utilities API – Complete Code Explanation (English + Interview Understanding)

Project Overview

This project is an Internal DevOps Utilities API developed using:

- Python
- FastAPI
- AWS Boto3
- psutil

The main goal of this project is:

 to centralize DevOps monitoring and infrastructure utility operations through REST APIs.

Instead of manually checking:

- CPU usage,
- memory usage,
- disk usage,
- and AWS resources,

this API provides a centralized backend utility system.

Project Flow

Client Request



FastAPI Router



Service Layer



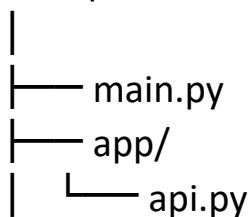
AWS/System Operations

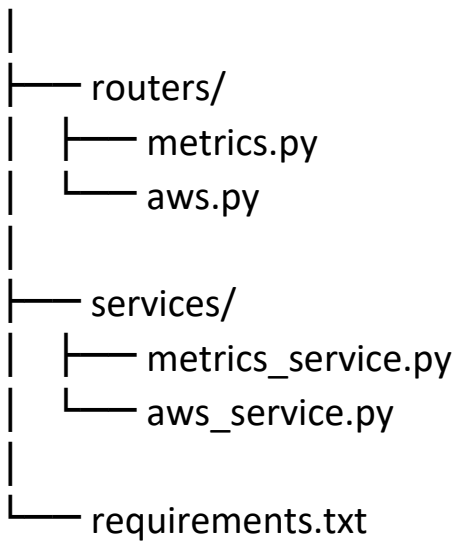


JSON Response

Folder Structure Understanding

DevOps-Utilities-API





Architecture Understanding

◇ routers/

This layer handles:

☞ API endpoints/routes.

Examples:

- /metrics
- /aws/s3

◇ services/

This layer contains:

☞ actual business logic.

Examples:

- CPU monitoring logic
- AWS bucket analysis logic

◇ app/api.py

This file:

☞ creates the FastAPI application.

◇ main.py

This file:

☞ starts the FastAPI server.

app/api.py Explanation

from fastapi import FastAPI

from routers import metrics, aws

◇ Import Statements

☑ FastAPI

Imports FastAPI framework.

FastAPI is used to:

👉 build REST APIs.

☑ routers import

from routers import metrics, aws

Imports:

- metrics routes
- AWS routes

into the application.

◇ Creating FastAPI App

```
app = FastAPI(  
    title="Internal DevOps Utilities API",  
    description="This is an Internal API Utilities App for Monitoring metrics, AWS Usage,  
Log Analysis, etc",  
    version="1.1.0",  
    doc_url="/docs",  
    redoc_url="/redoc"  
)
```

🧠 Explanation

This creates the FastAPI application.

◇ title

Defines API name.

◇ description

Explains what the API does.

◇ version

API version number.

◇ doc_url="/docs"

Enables Swagger API documentation.

Open:

localhost:8000/docs

to access interactive API docs.

◇ **redoc_url="/redoc"**

Alternative API documentation UI.

◇ **Root Endpoint**

```
@app.get("/")  
def hello():
```

Explanation

This creates:

👉 root endpoint.

If user calls:

/

API returns:

```
return {"message": "Hello Dosto, This is DevOps Utilites API"}
```

JSON response.

◇ **Include Routers**

```
app.include_router(metrics.router)  
app.include_router(aws.router, prefix="/aws")
```

Explanation

This attaches routers to the FastAPI app.

◇ **metrics.router**

Enables metrics APIs.

◇ **aws.router**

Enables AWS APIs.

Using prefix:

/aws

Final endpoints become:

- /aws/s3
- /aws/ec2

routers/metrics.py Explanation

```
from fastapi import APIRouter, HTTPException
```

```
from services.metrics_service import get_system_metrics
```

◇ **APIRouter**

Used for organizing API routes.

◇ **HTTPException**

Used for API error handling.

◇ **Import service**

Imports business logic function.

◇ **Create Router**

```
router = APIRouter()  
Creates router object.
```

◇ **Metrics Endpoint**

```
@router.get("/metrics",status_code=200)  
def get_metrics():
```



Explanation

Creates endpoint:
/metrics

◇ **Try Block**

```
metrics = get_system_metrics()  
return metrics  
Calls service layer function.
```

◇ **Except Block**

```
raise HTTPException(  
    status_code=500,  
    detail="Internal Server Error"  
)
```

If error occurs:



API returns HTTP 500.



services/metrics_service.py Explanation

import psutil

◇ **psutil**

Python library for:

- CPU monitoring
- memory monitoring

- disk monitoring

◊ **Function**

```
def get_system_metrics():  
Collects system metrics.
```

◊ **CPU Usage**

```
cpu_percent = psutil.cpu_percent(interval=1)  
Gets CPU usage percentage.
```

◊ **Memory Usage**

```
memory_percent = psutil.virtual_memory().percent  
Gets RAM usage percentage.
```

◊ **Disk Usage**

```
disk_percent = psutil.disk_usage("/").percent  
Gets disk storage usage percentage.
```

◊ **CPU Threshold**

```
cpu_threshold = 10  
Defines CPU threshold value.
```

◊ **Health Status Logic**

```
status = "High CPU" if cpu_percent > cpu_threshold else "Healthy"
```

Explanation

If:

CPU > 10

Then:

High CPU

Otherwise:

Healthy

◊ **Return JSON Response**

```
return {  
    "cpu_percentage":cpu_percent,  
    "memory_percentage":memory_percent,  
    "disk_percentage":disk_percent,  
    "cpu_threshold":cpu_threshold,  
    "system_status":status
```

```
}
```

Returns metrics as JSON response.

routers/aws.py Explanation

```
from services.aws_service import get_bucket_info
```

Imports AWS business logic.

◇ **S3 Endpoint**

```
@router.get("/s3",status_code=200)
```

```
def get_buckets():
```

Creates endpoint:

```
/aws/s3
```

◇ **Function Call**

```
buckets_info = get_bucket_info()
```

Calls AWS analysis logic.

services/aws_service.py Explanation

```
import boto3
```

Imports AWS SDK.

◇ **Create S3 Client**

```
s3_client = boto3.client("s3")
```

Creates AWS S3 connection.

◇ **List Buckets**

```
buckets = s3_client.list_buckets()["Buckets"]
```

Fetches all S3 buckets from AWS account.

◇ **Current Date**

```
current_date = datetime.now(timezone.utc).astimezone()
```

Gets current timezone-aware date.

◇ **New & Old Bucket Lists**

```
new_buckets = []
```

```
old_buckets = []
```

Used for bucket classification.

◇ **Loop Through Buckets**

```
for bucket in buckets:
```

Iterates through all buckets.

◇ **Bucket Name**

```
bucket_name = bucket["Name"]
```

Gets bucket name.

◇ **Creation Date**

```
creation_date = bucket["CreationDate"]
```

Gets bucket creation date.

◇ **90 Days Logic**

```
days_ago_90 = current_date - timedelta(days=90)
```

Calculates date before 90 days.

◇ **Bucket Classification**

```
if creation_date < days_ago_90:
```

If bucket older than 90 days:

➞ old bucket.

Otherwise:

➞ new bucket.

◇ **Final JSON Response**

```
return {  
    "total_buckets":len(buckets),  
    "new_buckets":len(new_buckets),  
    "old_buckets":len(old_buckets),  
}
```

Returns bucket analysis response.

💧 **main.py Explanation**

```
from app.api import app
```

```
import uvicorn
```

◇ **uvicorn**

ASGI server for FastAPI.

Used to:

➞ run FastAPI application.

◇ **Main Entry Point**

```
if __name__ == "__main__":
```


Runs only when file executed directly.

◇ **Run Server**

```
uvicorn.run(  
    "app.api:app",  
    host="0.0.0.0",  
    port=8000,  
    reload=True  
)
```

Explanation

◇ **host="0.0.0.0"**

Makes API accessible on network.

◇ **port=8000**

Application runs on port 8000.

◇ **reload=True**

Automatically reloads server on code changes.
Developer-friendly feature.

requirements.txt Explanation

fastapi

uvicorn

psutil

boto3

Dependency Understanding

fastapi

REST API framework.

uvicorn

ASGI web server.

psutil

System monitoring library.

boto3

AWS SDK for Python.

Real DevOps Concepts Used

- Infrastructure Monitoring
- System Health Analysis

- Cloud Resource Monitoring
- REST API Development
- Backend Modular Architecture
- AWS Integrations
- DevOps Automation

BEST INTERVIEW CLOSING LINE

“This project helped me understand production-style FastAPI backend architecture, infrastructure monitoring, AWS integrations, REST APIs, and DevOps automation workflows through practical implementation.”